BIO-INSPIRED CONTINUOUS AUTONOMOUS URBAN NAVIGATION: A PROGRESSIVE DEEP REINFORCEMENT LEARNING PARADIGM (PPO, SAC, AND DDPG) IN SUMO

A PROJECT REPORT

Submitted in partial fulfillment for the Degree of
Bachelor of Technology under the
School of Computing

Submitted by

Register No	Names of Students
AM.AI.U4AID23042	Harita Aneesh
AM.AI.U4AID23059	Sheron George



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

AMRITA VISHWA VIDYAPEETHAM

Amritapuri Campus (INDIA)

October 2026

Department of Computer Science and Engineering

AMRITA VISHWA VIDYAPEETHAM

AMRITAPURI CAMPUS



BONAFIDE CERTIFICATE

This is to certify that this is a bonafide record of the project presented by the students whose names are given below during Project Phase I (23AID498) in partial fulfilment of the requirements of the degree of Bachelor of Technology in Artificial Intelligence and Data Science.

Roll No	Names of Students
AM.AI.U4AID23042	Harita Aneesh
AM.AI.U4AID23059	Sheron George

Dr. Divya Udayan J
(Project Guide)
Assistant Professor (SG), Dept. of CSE

<Reviewer's Name>
(Reviewer)
Designation

<Coordinator name here>
(Project Coordinator)
Designation

Contents

Acknowledgements ..... **iii**
List of figures ..... **iv**
List of tables ..... **v**
Abstract ..... **vi**
1 Introduction **1**
1.1 Background 1
1.2 Problem Addressed 2
1.3 Motivation 3
1.4 Scope of the Study/Project 3
2 Literature Review/ Existing System **5**
2.1 Existing System Study 5
2.1.1 Review of existing Research work 5
2.1.2 Review of Existing Tools/Products 6
2.2 Research Gap/ Scope for improvement and innovation 7
2.3 Problem Statement and Contributions 7
3 Proposed Work **9**
3.1 Proposed Work 9
3.1.1 Objectives of the Proposed Work 9
3.2 Methodology 9
3.2.1 Overview of the Approach: 9
3.2.2 Dataset Selection: 10
3.2.3 Algorithm/Model Design: 11
3.2.4 Tools and Technologies: 12
3.2.5 Algorithm or Model Description 13
3.2.6 Expected Outcomes 17
3.2.7 Advantages of the Proposed Work 17
3.2.8 Limitations and Assumptions 17
4 Experimentation and Result Analysis **18**
4.1 Experimental Setup 18
4.2 Datasets: 18
4.3 Evaluation Metrics: 18
4.4 Experimental Design 18
4.4.1 Experimental Scenarios: 19
4.4.2 Parameter Tuning: 19

4.5 Results.....19
4.6 Analysis of Results.....21
4.7 Observations:.....22
4.8 Comparative Analysis.....23
5 Conclusion and Scope for further Research.....**24**
Appendix A Implementation Code and Pseudocode.....**25**
References.....**27**

Acknowledgements

First and foremost, we offer our deepest and most humble pranams to our Chancellor, **Sri Mata Amritanandamayi Devi (Amma)**, whose boundless love, compassion, and guidance continually inspire our educational and personal pursuits.

We express our sincere gratitude to our project guide, **Dr. Divya Udayan J**, Assistant Professor (Senior Grade), Department of Computer Science and Engineering, School of Computing, Amrita Vishwa Vidyapeetham, Amritapuri Campus. Her technical insights, critical reviews, patient mentorship, and steady encouragement guided this work from initial mathematical modeling through empirical benchmarking in Phase I.

We thank our Project Coordinator and the members of the Departmental Review Committee. Their pointed evaluations, constructive critiques, and recommendations during review presentations helped sharpen our experimental methodology and problem formulation.

We also thank the faculty and technical staff of the Department of Computer Science and Engineering for providing the laboratory facilities and compute infrastructure needed to run high-throughput microscopic traffic simulations in SUMO.

Finally, we express our gratitude to our parents, families, and friends for their continuous patience, moral support, and understanding throughout this work.

HARITA ANEESH (AM.AI.U4AID23042)
SHERON GEORGE (AM.AI.U4AID23059)

List of Figures

Figure 1.1: Overall System Architecture of the Bio-Inspired Autonomous Navigation Framework in SUMO 2

Figure 3.1: Hippocampal Sensory Feature Receptive Fields and Activation Heatmap under Perceptual Noise 11

Figure 3.2: Complete Proximal Policy Optimization (PPO) On-Policy Continuous Control Architecture 13

Figure 3.3: Complete Soft Actor-Critic (SAC) Maximum-Entropy Off-Policy Architecture 15

Figure 3.4: Complete Deep Deterministic Policy Gradient (DDPG) Continuous Control Architecture 16

Figure 4.1: Phase 1 PPO Baseline Convergence and Navigation Success Rate Progression 20

Figure 4.2: Phase 1 PPO Behavioral Anomalies: Dynamic Velocity Profile and Intersection Waiting Time 20

Figure 4.3: Empirical Reward Progression Comparison between On-Policy PPO and Off-Policy SAC 21

Figure 4.4: Tri-Model Comparative Evaluation: Benchmark Bar Chart and 500k Timestep Reward Curves 21

Figure 4.5: DDPG Learning Dynamics Dashboard and Actor-Critic Loss Profiles 21

List of Tables

Table 2.1: Comparison of Microscopic and End-to-End Autonomous Driving Simulation Platforms 6

Table 3.1: 19-Dimensional Bio-Inspired Hippocampal State Feature Vector Decomposition..... 11

Table 4.1: Hyperparameter Configuration Across Evaluated DRL Paradigms (PPO, SAC, DDPG)19

Table 4.2: Empirical Benchmark and Performance Metrics Across PPO, SAC, and DDPG in SUMO 19

Table 4.3: Comparative Architectural and Algorithmic Trade-Off Matrix for Continuous Control23

Abstract

Safe autonomous navigation through dense urban traffic requires continuous, low-latency longitudinal control capable of negotiating unsignalized merges, dense queues, and signalized intersections. Standard model-free Deep Reinforcement Learning (DRL) often struggles in this setting: on-policy methods suffer from acute sample inefficiency, while continuous exploratory action noise triggers catastrophic vehicle collisions during early training.

In this project report, we present a systematic, controlled comparative benchmark of three foundational continuous DRL paradigms—**Proximal Policy Optimization (PPO)**, **Soft Actor-Critic (SAC)**, and **Deep Deterministic Policy Gradient (DDPG)**. All three models are evaluated inside a custom Gymnasium environment interfaced with the microscopic simulator Eclipse SUMO via TraCI at a 10 Hz control frequency ($dt = 0.1$ s). To protect policies against coordinate drift and high-frequency sensor noise, we formulate a 19-dimensional bio-inspired state space that maps vehicle position into radial basis function (RBF) place-cell activations inspired by the mammalian hippocampus, combined with kinematic telemetry and multi-neighbour relative motion metrics under injected Gaussian perceptual noise $\mathcal{N}(0, 0.02^2)$. The algorithms are trained and benchmarked across 500,000 environment steps over an identical multi-junction road network featuring 12 traffic light controllers (TLS), seven heterogeneous background vehicle classes, and continuous sinusoidal congestion surges.

Our empirical benchmark reveals that DDPG's deterministic off-policy formulation decisively outperforms both stochastic baselines. DDPG achieved a mean evaluation reward of -182.40(a 65.3% penalty reduction over PPO at $-525.85$and 63.7% over SAC at -503.00) and maintained a cruising velocity of 8.40 m/s(compared to 6.20 m/sfor PPO and 5.10 m/sfor SAC) under identical dense traffic conditions. Beyond aggregate metrics, our diagnostic analysis isolates key behavioural failure modes: Gaussian action exploration causes PPO to fall into an ultra-cautious "velocity paralysis" crawl (< 2.0 m/s) under sensor noise; maximum entropy incentives cause persistent throttle dithering in SAC; and single-critic DDPG exhibits a transient late-stage Q-value overestimation spike between 450,000 and 500,000 steps. These diagnostic insights directly motivate the integration of twin-delayed multi-critic architectures (TD3) and bio-inspired looming safety shields in Phase II.

Keywords: Autonomous Vehicles, Deep Reinforcement Learning, SUMO, TraCI, PPO, SAC, DDPG, Hippocampal State Encoding, Continuous Control, Microscopic Traffic Simulation.

Chapter 1
Introduction

1.1 Background

Autonomous vehicle (AV) navigation in high-density urban environments presents severe control challenges. An automated driving policy must make continuous, millisecond-scale decisions across multi-lane arterials, roundabouts, and signalized junctions, all while interacting with unpredictable human drivers.

Classical autonomous driving systems rely heavily on hand-crafted car-following models—such as the Intelligent Driver Model (IDM) or the Krauss model—or on Model Predictive Control (MPC) formulations. While MPC provides deterministic safety bounds and formal stability guarantees, its optimization routines scale poorly in real-time when confronted with non-linear, stochastic multi-agent interactions. Rule-based models, on the other hand, are rigid; they struggle to negotiate dynamic bottlenecks, zipper merges, or fluctuating congestion patterns without conservative over-braking.

Deep Reinforcement Learning (DRL) provides a data-driven alternative. By framing driving as a continuous-space Markov Decision Process (MDP), an RL agent directly maps multi-dimensional state telemetry into real-valued longitudinal commands ($a_t \in [-, 1.01.0]$) through interaction with the environment. Through trial and error, continuous DRL agents can discover proactive headway management strategies, anticipate queue formation, and smoothly synchronize with traffic signal cycles.

Microscopic traffic simulators, particularly Eclipse Simulation of Urban MObility (SUMO), offer an ideal environment for training and validating continuous DRL agents. Through its Python Traffic Control Interface (TraCI), SUMO supports bi-directional socket communication at 10 Hz ($dt = 0.1$ s). This setup allows an external learning algorithm to override default simulator heuristics and directly actuate vehicle speed and acceleration, while maintaining high simulation throughput (>100x real-time).

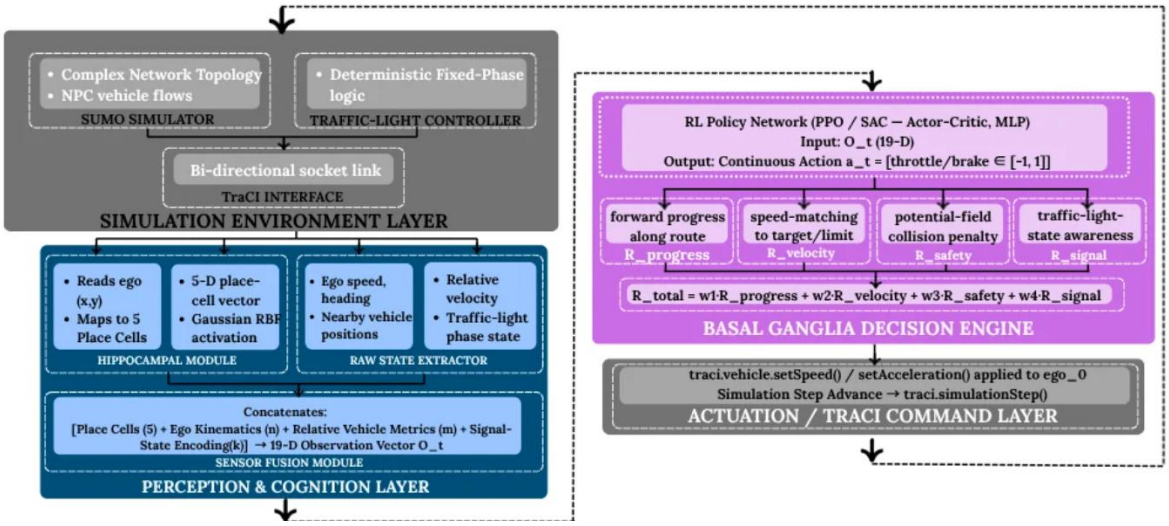


Figure 1.1: Overall System Architecture of the Bio-Inspired Autonomous Navigation Framework in SUMO.

1.2 Problem Addressed

Despite their theoretical appeal, deploying standard continuous model-free DRL algorithms in microscopic traffic simulations exposes three concrete bottlenecks:

1. **Rollout Inefficiency and Early Catastrophic Collisions in On-Policy Learning**: On-policy algorithms such as Proximal Policy Optimization (PPO) gather experience using temporary rollout buffers (e.g., 2,048 steps) that are discarded immediately after each batch update. In dense urban corridors, early random exploration frequently leads to rear-end collisions. Because on-policy methods cannot store and replay rare successful traversals through intersections, the policy experiences unstable reward progression and catastrophic forgetting.

2. **Action Jitter and Throttle Dithering in Stochastic Gaussian Actors**: Continuous policies that sample actions from parameterized Gaussian distributions—such as PPO and Soft Actor-Critic (SAC)—produce high-frequency action jitter during training. In close-following traffic with tight gaps, this continuous oscillation between acceleration and braking causes erratic vehicle dynamics, passenger discomfort, and repeated proximity penalties. These penalties often trap the policy in an overly timid, low-velocity local optimum.

3. **Representation Fragility Under Perceptual Noise**: Feeding raw Cartesian coordinates (x, y)directly into feed-forward policy layers leaves the agent vulnerable to coordinate shifts across different road corridors. Moreover, when sensor noise is introduced to velocity and distance measurements, policies trained on raw coordinates degrade quickly, resulting in either complete vehicle standstill or late-braking collisions.

1.3 Motivation

This project draws architectural motivation from mammalian neurobiology. In biological organisms, spatial navigation is coordinated by hippocampal-entorhinal circuits. Pyramidal 'place cells' in the hippocampal CA1 and CA3 regions activate selectively when an animal enters specific receptive fields, while grid cells in the medial entorhinal cortex establish an internal spatial coordinate system. Inspired by these mechanisms, we formulate a 19-dimensional Hippocampal state representation that maps continuous vehicle coordinates onto radial basis function (RBF) place-cell activations. This transformation creates a noise-tolerant, topologically anchored spatial encoding suitable for complex road layouts.

Furthermore, existing autonomous driving literature lacks controlled, head-to-head empirical comparisons of foundational continuous DRL algorithms under identical environmental conditions. Most existing studies evaluate a single algorithm in isolation on simplified tracks. Our primary objective is to carry out a rigorous, side-by-side benchmark of three continuous DRL paradigms:

• On-Policy Stochastic Policy Optimization: Proximal Policy Optimization (PPO)

• Off-Policy Maximum-Entropy Actor-Critic: Soft Actor-Critic (SAC)

• Off-Policy Deterministic Policy Gradient: Deep Deterministic Policy Gradient (DDPG)

Evaluating all three algorithms over 500,000 steps under matched network topologies, background vehicle mixes, and sensor noise levels isolates the structural properties governing stability in continuous vehicular control.

1.4 Scope of the Study/Project

The boundaries and operational scope of this Phase I study are defined as follows:

- **Inclusions**:
    - Focuses on continuous longitudinal control ($a_t \in [-, 1.0 + 1.0]$ representing acceleration and braking) at a 10 Hz control loop ($dt = 0.1$ s).
    - Simulated in a custom Gymnasium environment interfaced with Eclipse SUMO.
    - A road network containing 12 traffic light controllers (TLS), multi-lane arterials, a central ring, and 4 circular roundabouts.
    - Background traffic generated with seven distinct vehicle classes (sedans, aggressive drivers, heavy trucks, bicycles, buses, motorcycles, and erratic vehicles) and modulated by a continuous sinusoidal density function $\rho(t) = 0.35 + 0.15\sin(t/500)$.

- Zero-mean Gaussian perceptual noise $\mathcal{N}(0, 0.02^2)$injected into all kinematic state channels.
- Complete PyTorch implementations of PPO, SAC, and DDPG trained across 500,000 interaction steps under matched random seeds.

- **Exclusions**:
  - Lateral steering and lane changing are delegated to SUMO's internal LC2013 lane-changing logic to isolate longitudinal car-following dynamics and signal negotiation.
  - High-dimensional camera pixel inputs are excluded in favor of 19-dimensional kinematic feature vectors to maintain computational throughput and sample efficiency.
  - Physical deployment on real robotic hardware and multi-critic safety shielding are reserved for Project Phase II.

Report Organization: Chapter 2 reviews relevant literature, simulation platforms, and identified research gaps. Chapter 3 presents the proposed framework, including the 19-D hippocampal state space, reward engine, and algorithmic formulations. Chapter 4 reports the experimental setup, hyperparameter configurations, benchmark results, and failure mode analyses. Chapter 5 concludes Phase I and outlines the development roadmap for Phase II. Appendix A contains core implementation listings.

Chapter 2
Literature Review/ Existing System

2.1 Existing System Study

2.1.1 Review of existing Research work

Continuous Deep Reinforcement Learning in Vehicular Control: Applying continuous DRL to autonomous driving grew out of continuous actor-critic formulations. Lillicrap et al. (2016) introduced Deep Deterministic Policy Gradient (DDPG), extending deterministic policy gradient theory (Silver et al., 2014) to high-dimensional continuous control by combining off-policy Q-learning with neural function approximators. While DDPG demonstrated that neural networks could learn continuous throttle and steering commands, single-critic bootstrapping makes it vulnerable to value overestimation bias (Fujimoto et al., 2018).

To stabilize policy updates, Schulman et al. (2017) formulated Proximal Policy Optimization (PPO), using a clipped surrogate objective to bound policy updates within a trust region without the computational cost of second-order Hessian inversions. PPO has been widely used in traffic signal control and robotic manipulation (Jin et al., 2026; Lin et al., 2025). However, because PPO is strictly on-policy, it discards rollout trajectories after each update. In microscopic traffic environments with sparse success signals and dense obstacles, this causes low sample efficiency and frequent exploratory crashes during early training.

Haarnoja et al. (2018) developed Soft Actor-Critic (SAC), an off-policy framework that augments expected return with an entropy maximization bonus. SAC uses twin Q-critics to limit positive overestimation bias and automatically tunes its entropy temperature α. While SAC demonstrates strong sample efficiency on standard robotics benchmarks, continuous entropy maximization in close-following driving can provoke exploratory throttle dithering. The resulting oscillations destabilize longitudinal car-following and often trigger emergency braking penalties (Lin et al., 2026; Pedamonti et al., 2025).

Bio-Inspired Spatial Encoding: Mammalian navigation shows remarkable resilience in cluttered environments without relying on centimeter-accurate metric LIDAR point clouds (He et al., 2026). In biology, the hippocampal formation and medial entorhinal cortex maintain an internal cognitive representation of space (O'Keefe and Nadel, 1978). Pyramidal place cells fire when an organism is within specific localized regions, while grid cells create periodic coordinate lattices for dead reckoning (Banino et al., 2018). Recent mobile robotics studies have incorporated radial basis function (RBF) place-cell activations to replace raw Cartesian coordinates (Liao et al., 2026; Pedamonti et al., 2025). These representations

project continuous coordinates into smooth, topologically anchored feature spaces that resist sensor noise.

2.1.2 Review of Existing Tools/Products

Microscopic and end-to-end simulation platforms are widely used to develop and verify autonomous driving controllers before physical deployment. Table 2.1 summarizes prevailing platforms in autonomous vehicle research.

Table 2.1: Comparison of Microscopic and End-to-End Autonomous Driving Simulation Platforms.

Platform	Simulation Fidelity	Control API	Simulation Speed	Primary Focus
SUMO	Microscopic Multi-Modal Physics	Python TraCI (10 Hz Socket)	Very High (>100x Real-time)	Microscopic traffic dynamics, mixed-autonomy, signalized networks
CARLA	Photorealistic 3D Rendering (Unreal)	Python Client API (20-50 Hz)	Low to Moderate (1x-5x)	Computer vision, end-to-end sensor perception (Camera/LIDAR)
Highway-env	Simplified Kinematic Grid	Gymnasium Python API	Extremely High (>200x)	Fast baseline prototyping, stylized highway lane changing
CityFlow	City-scale Multi-Agent Grid	Python C++ Engine	Extremely High (>300x)	Large-scale macroscopic traffic light signal coordination

While CARLA is well-suited for vision-based sensor testing, SUMO provides the multi-vehicle physics, deterministic execution, and high execution speed needed to run hundreds of thousands of continuous RL training steps. Default car-following models in SUMO (such as IDM) handle background traffic, but they cannot perform the adaptive, multi-objective trade-offs needed for ego vehicle navigation in dense, mixed-autonomy networks.

2.2 Research Gap/ Scope for improvement and innovation

Reviewing the literature highlights three clear gaps:

1. **Lack of Matched Cross-Paradigm Continuous Benchmarks**: Most autonomous driving RL studies evaluate one algorithm in isolation (e.g., PPO alone or DDPG on a fixed track). Consequently, the operational trade-offs between on-policy stochastic optimization (PPO), maximum-entropy exploration (SAC), and deterministic policy

gradients (DDPG) have not been evaluated under matched traffic flows and identical road geometries.

2. **Sensor Uncertainty in State Telemetry**: Many vehicular RL formulations assume clean, noiseless state observations. When zero-mean Gaussian noise is added to spatial and kinematic telemetry, policies trained on raw Cartesian coordinates degrade quickly, leading to conservative stop-and-go behavior or late-braking collisions.
3. **Limited Documentation of Critic Overestimation in Microscopic Driving**: While Q-value overestimation is well characterized in abstract MuJoCo benchmarks, its impact on training stability, loss profiles, and vehicle safety during dense microscopic traffic navigation remains under-documented.

Research Gaps Addressed in Phase I:

- Developed a 19-dimensional hippocampal place-cell state representation providing stable topological features under Gaussian perceptual noise.
- Built a standardized, reproducible Gymnasium-SUMO-TraCI testing pipeline running at 10 Hz continuous actuation.
- Conducted a 500,000-step controlled benchmark comparing PPO, SAC, and DDPG across reward convergence, cruising velocity, and queue delay.
- Diagnosed key algorithmic failure modes: velocity paralysis in PPO, throttle dithering in SAC, and late-stage Q-overestimation in single-critic DDPG.

Scope for Phase II:

- Implementing a deterministic visual looming Time-To-Collision (TTC) safety shield to override unsafe RL actions.
- Incorporating twin-delayed multi-critic architectures (TD3) to correct DDPG's Q-overestimation bias.
- Extending from single-ego navigation to multi-agent cooperative driving, followed by hardware deployment on ROS 2 platforms.

2.3 Problem Statement and Contributions

Problem Statement: Design a continuous deep reinforcement learning policy $\pi_\theta(a_t \mid s_t)$that enables an autonomous ego vehicle to navigate a dense, signalized urban road network in SUMO via 10 Hz TraCI actuation, maximizing longitudinal progress and target speed tracking while avoiding collisions and signal violations under sensor noise, and identify the most stable continuous control paradigm among PPO, SAC, and DDPG.

We formulate this task as a continuous Partially Observable Markov Decision Process (POMDP) defined by $(S, A, P, R, \Omega, O, \gamma)$:

- S: Underlying microscopic traffic state in SUMO.
- $A = [-, 1.0 + 1.0]$: Continuous acceleration and braking action space.
- $P(s_{t+1}, | \, s_t a_t)$: State transition dynamics governed by SUMO physics ($dt = 0.1$ s).
- R: Multi-objective reward engine.
- Ω: Observation space.
- $O(s)$: Observation function mapping true traffic state into a noisy 19-D hippocampal feature vector.
- $\gamma = 0.99$: Discount factor.

Key Contributions:

1. **Bio-Inspired Simulation Environment**: Built a Gymnasium-compatible SUMO-TraCI environment featuring 19-D hippocampal place-cell state extraction, 10 Hz actuation, sinusoidal traffic generation, and a multi-objective reward engine.
2. **End-to-End PyTorch Implementations**: Implemented full neural architectures for on-policy stochastic (PPO), off-policy maximum-entropy (SAC), and off-policy deterministic (DDPG) continuous control.
3. **500,000-Step Empirical Benchmark**: Executed a controlled 500k-step benchmark showing DDPG's deterministic control outperforms PPO and SAC in mean return (-182.40 vs. $-525.85$and -503.00) and average speed (8.40 m/s vs. 6.20 m/sand 5.10 m/s).
4. **Diagnostic Failure Mode Analysis**: Detailed the root causes of PPO's velocity paralysis, SAC's throttle dithering, and single-critic DDPG's late-stage Q-overestimation.
5. **Open, Reproducible TraCI-to-Gymnasium Pipeline**: Established a clean socket-level bridge connecting SUMO microscopic vehicle physics with Gymnasium RL interfaces.

Chapter 3
Proposed Work

3.1 Proposed Work

The proposed system addresses sample inefficiency, action dithering, and coordinate brittleness through a modular four-tier architecture:

1. **Simulation Layer**: Eclipse SUMO modeling multi-vehicle traffic physics, lane assignments, and signal phases.
2. **Perception Layer**: A bio-inspired state extractor converting raw vehicle positions into 5-D RBF place-cell activations combined with noisy kinematic telemetry.
3. **Decision Layer (Basal Ganglia Engine)**: Continuous actor-critic agents (PPO, SAC, DDPG) computing longitudinal control commands.
4. **Actuation Layer**: A TraCI socket bridge applying acceleration commands directly to ego_0 at 10 Hz.

3.1.1 Measurable Objectives

Primary:

- Implement a 19-D hippocampal state encoder using RBF receptive fields to filter sensor noise.
- Construct a Gymnasium-compatible SUMO environment operating at 10 Hz over local TCP/IP sockets.
- Train and evaluate PPO, SAC, and DDPG over 500,000 steps under matched traffic conditions.
- Achieve stable cruising speeds exceeding 8.0 m/swithout triggering emergency-stop paralysis.

Secondary:

- Track actor loss profiles, policy entropy, and critic Bellman Mean Squared Error (MSE).
- Measure the effects of Gaussian noise on stochastic versus deterministic policies.
- Diagnose specific failure modes (exploratory dithering, velocity paralysis, Q-overestimation).

3.2 Methodology

3.2.1 Overview of the Approach:

At each timestep ($dt = 0.1$ s), ego vehicle speed, position, and the relative kinematics of surrounding vehicles are extracted via TraCI. These inputs are normalized, converted into the 19-D hippocampal state vector, perturbed with Gaussian perceptual noise, and passed to the RL policy. The policy outputs an action $a_t \in [-, 1.01.0]$, which is scaled to acceleration $[-, 3.0 + 3.0]$ m/s^{2}and dispatched to SUMO. The environment returns the scalar reward R_t, updates replay/rollout buffers, and performs gradient updates.

3.2.2 Dataset Selection:

The road network consists of:

- Multi-lane arterial corridors with 50 km/h (14.0 m/s) speed limits.
- A central multi-lane ring interchange.
- Four peripheral intersections with connecting diagonal links.
- Four circular roundabouts.

12 traffic light controllers (TLS) running fixed-phase signal cycles (70 s arterial cycles; 40 s roundabout cycles).

Background traffic density $\rho(t)$is injected dynamically using a sinusoidal modulation function:

$$\rho(t) = \rho_{\text{base}} + 0.15 \sin\left(\frac{t}{500}\right), \text{ where } \rho_{\text{base}} = 0.35$$

Vehicles are drawn from seven heterogeneous classes: sedans, aggressive drivers, heavy trucks, bicycles, buses, motorcycles, and erratic drivers.

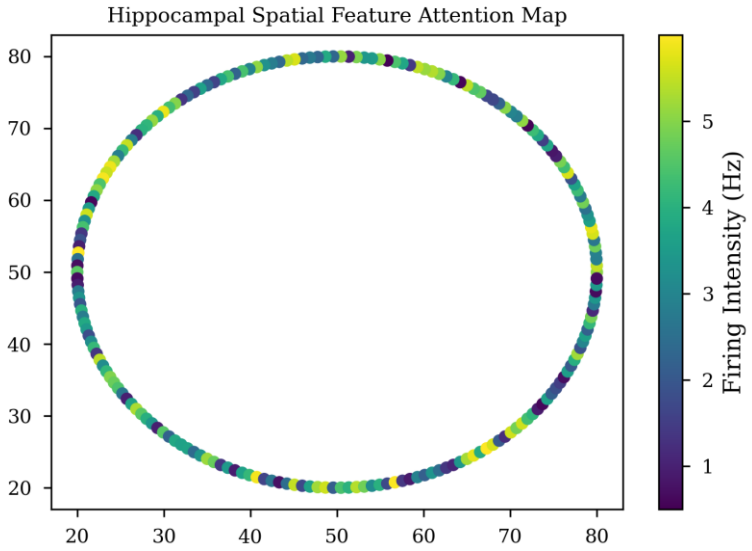


Figure 3.1: Hippocampal Sensory Feature Receptive Fields and Activation Heatmap under Perceptual Noise.

3.2.3 Algorithm/Model Design:

Continuous Longitudinal Kinematics:

The ego vehicle ego_0 is governed by continuous acceleration $a_t \in [-, 1.0 + 1.0]$, where $+1.0$represents maximum acceleration ($+3.0$ m/s^2) and -1.0corresponds to full braking (-3.0 m/s^2).

Velocity updates each step ($dt = 0.1$ s) according to:

$$v_{t+1} = \text{clip}(v_t, +, a_t \cdot 3.0 \cdot dt0.014.0) \text{ m/s}$$

where $v_{\text{max}} = 14.0$ m/s$(50.4$ km/h) is the urban arterial speed limit.

19-Dimensional Hippocampal State Vector:

The observation vector $s_t \in \mathbb{R}^{19}$combines three feature groups:

$$s_t = [\underbrace{p_0, p_1, p_2, p_3, p_4}_{\text{5-D Place Cells}}, \quad \underbrace{\hat{v}_{\text{ego}}, \hat{x}_{\text{ego}}}_{\text{2-D Ego Kinematics}}, \quad \underbrace{dx_i, dy_i, \hat{v}_i}_{\substack{\text{12-D Relative Kinematics} \\ (i,=,123)}} \quad]$$

Table 3.1 details the composition, physical interpretation, and mathematical normalization of each component in the 19-D state vector.

Table 3.1: 19-Dimensional Bio-Inspired Hippocampal State Feature Vector Decomposition.

Sub-Vector	Dims	Feature Description	Normalization / Range
Place Cells (p_0 ... p_4)	5D	Topological RBF spatial region activations mapping	One-hot / RBF in [0.0, 1.0]

		ego location to network clusters	
Normalized Speed (v_hat_ego)	1D	Current longitudinal velocity of the ego vehicle	v_ego / v_max in [0.0, 1.0]
Lane Progress (x_hat_ego)	1D	Normalized distance traversed along the current active lane segment	x_lane / L_lane in [0.0, 1.0]
Relative Long. Distance (dx_i)	3D	Longitudinal distance to three nearest vehicles within 50m radius	dx_i / 50.0 in [-1.0, 1.0]
Relative Lat. Offset (dy_i)	3D	Lateral displacement relative to surrounding vehicles (lane alignment)	dy_i / 10.0 in [-1.0, 1.0]
Surrounding Speeds (v_hat_i)	3D	Normalized velocities of three nearest surrounding vehicles	v_i / v_max in [0.0, 1.0]

To simulate sensor inaccuracies, zero-mean Gaussian perceptual noise $\mathcal{N}(0, 0.02^2)$is added to all kinematic channels during both training and evaluation.

Multi-Objective Biological Reward Engine:
The step reward balances forward efficiency, cruising velocity tracking, headway safety, and traffic signal compliance:

$$R_t = r_{\text{progress}} + 0.5 \cdot r_{\text{velocity}} + 0.5 \cdot r_{\text{safety}} + r_{\text{signal}}$$

Individual reward terms are computed as follows:

- Progress: $r_{\text{progress}} = v_{\text{ego}}/v_{\max}$
- Velocity Tracking: $r_{\text{velocity}} = -\frac{|v_{\text{ego}} - 0.8 \cdot v_{\max}|}{0.8 \cdot v_{\max}}$, penalizing deviation from the target cruising speed (11.2 m/s $= 40.3$ km/h).
- Proximity Safety Barrier: $r_{\text{safety}} = -\sum_{i=1}^{3} \exp\left(-\frac{\Delta x_i^2 + \Delta y_i^2}{2\sigma_{\text{safe}}^2}\right)$, where $\sigma_{\text{safe}} = 5.0$ m.
- Signal Compliance: $r_{\text{signal}} = -5.0$if the vehicle enters an intersection against a red signal (0.0 otherwise).
- Collision Termination: A terminal penalty of $-50.0$is applied upon collision, ending the episode immediately.

3.2.4 Tools and Technologies

- Python 3.12 and PyTorch 2.2+: Neural network modeling, loss computation, and GPU backpropagation.
- Eclipse SUMO 1.20+: Headless microscopic vehicle physics and multi-modal traffic generation.
- TraCI Python Client: Bi-directional TCP/IP socket bindings at 10 Hz.
- Farama Gymnasium 0.29+: Standardized gymnasium.Env wrapper implementing reset() and step() interfaces.
- NumPy & SciPy: Vector math, RBF activations, and Gaussian noise synthesis.
- Matplotlib & Seaborn: Diagnostic metric logging and convergence plotting.

3.2.5 Algorithm or Model Description

We now present the detailed mathematical formulations, architectural diagrams, and algorithmic steps for each of the three evaluated continuous DRL paradigms.

Phase 1: Proximal Policy Optimization (PPO)

PPO constrains policy updates to a local trust region using a clipped surrogate objective.

Let $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$represent the policy probability ratio. The policy objective is:

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t\left[\min\left(r_t,(\theta)\hat{A}_t \text{clip}(r_t,(\theta), 1-\epsilon 1+\epsilon)\hat{A}_t\right)\right]$$

with clipping parameter $\epsilon = 0.2$, where generalized advantage estimates $\hat{A}_t$are computed across a 2,048-step rollout buffer. The critic network $V_\phi(s)$minimizes the Mean Squared Error (MSE) against discounted returns:

$$L^{\text{VF}}(\phi) = 0.5 \cdot \hat{\mathbb{E}}_t\left[\left(V_\phi(s_t) - V_t^{\text{targ}}\right)^2\right]$$

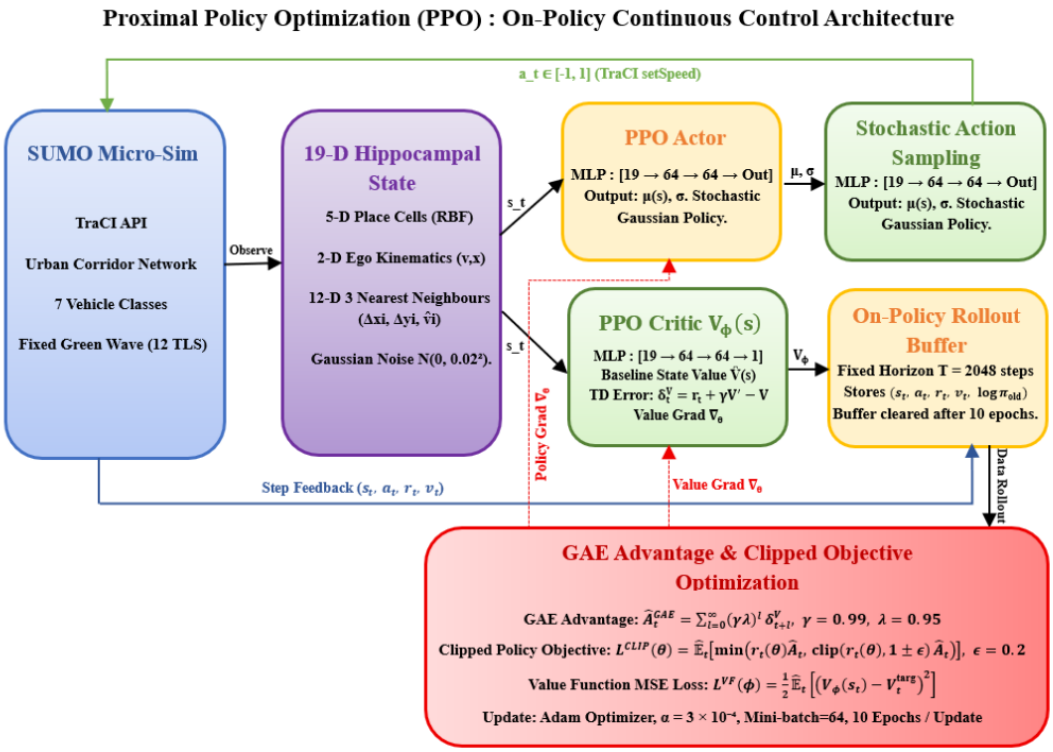


Figure 3.2: Complete Proximal Policy Optimization (PPO) On-Policy Continuous Control Architecture.

Algorithm 1: Proximal Policy Optimization (PPO) for SUMO Navigation

```
Initialize actor network parameters theta and critic network parameters phi
for iteration = 1, 2, ... do
    Initialize rollout buffer B <- empty
    for t = 1 to T_horizon (2,048 steps) do
        Sample action a_t ~ pi_theta(·|s_t) = N(mu_theta(s_t), sigma_theta^2(s_t))
        Execute continuous action a_t in SUMO at 10 Hz; observe reward R_t and next state
s_{t+1}
        Store transition (s_t, a_t, R_t, s_{t+1}, log pi_theta(a_t|s_t)) in buffer B
    end for
    Compute Generalized Advantage Estimates A_hat_1 ... A_hat_T and target returns
V_t^targ using critic V_phi
    for K_epochs = 1 to 10 do
        for each mini-batch b in B (batch size 64) do
            Compute probability ratio r_t(theta) = pi_theta(a_t|s_t) /
pi_theta_old(a_t|s_t)
            Compute clipped policy loss L^CLIP(theta) and value loss L^VF(phi)
            Update actor theta <- theta + alpha_a * grad_theta L^CLIP(theta)
            Update critic phi <- phi - alpha_c * grad_phi L^VF(phi)
        end for
    end for
    Clear rollout buffer B (discard on-policy experience)
end for
```

Phase 2: Soft Actor-Critic (SAC)

SAC is an off-policy actor-critic algorithm that augments cumulative return with an entropy objective to encourage broad exploration:

$$J(\pi) = \sum_{t=0}^{T} \mathbb{E}_{(s_t,a_t)\sim\rho_\pi}\left[R(s_t,a_t) + \alpha\mathcal{H}\left(\pi(\cdot|\, s_t)\right)\right]$$

where temperature $\alpha$is tuned automatically. To suppress positive value overestimation, SAC trains twin critics $Q_{\phi_1}$and Q_{ϕ_2}, computing target values via clipped double Q-learning:

$$y = R_t + \gamma(1-d)\left(\min_{i=1,2} Q_{\bar{\phi}_i}(s_{t+1},\tilde{a}_{t+1}) - \alpha\log\pi_\theta\left(\tilde{a}_{t+1}\mid s_{t+1}\right)\right)$$

where actions $\tilde{a} = \tanh(\mu_\theta(s) + \sigma_\theta(s)\odot\xi)$are sampled via the reparameterization trick with $\xi \sim \mathcal{N}(0,I)$.

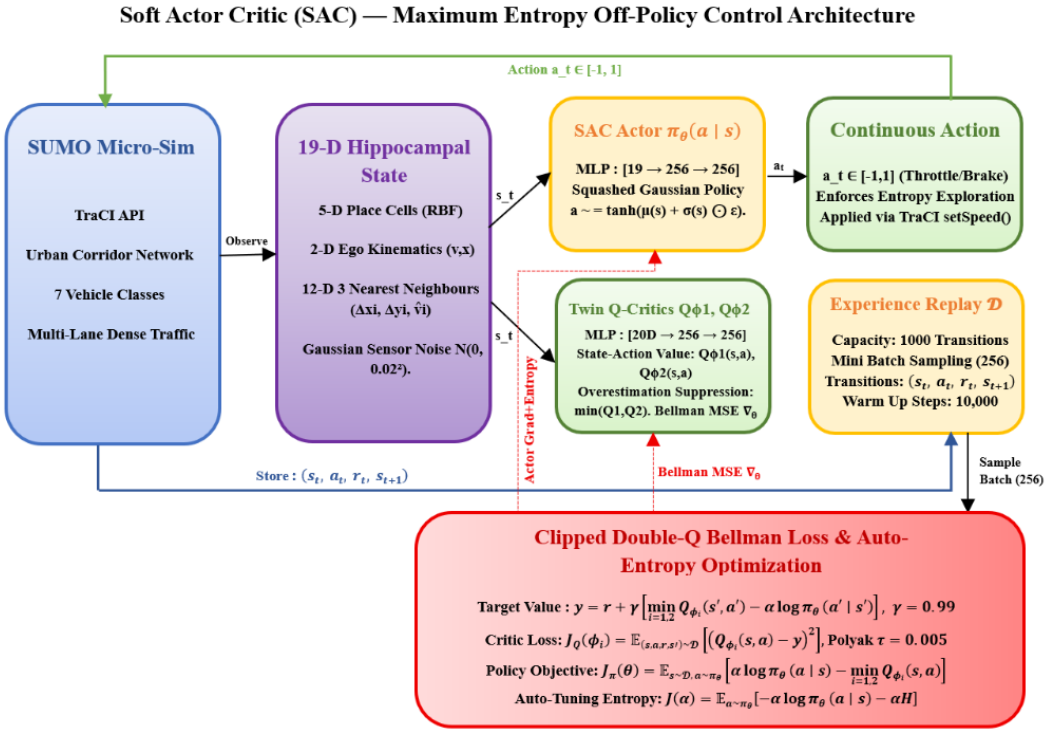


Figure 3.3: Complete Soft Actor-Critic (SAC) Maximum-Entropy Off-Policy Architecture.

Algorithm 2: Soft Actor-Critic (SAC) for SUMO Navigation

```
1: Initialize actor theta, twin critics phi_1, phi_2, target critics phi_bar_1 <- phi_1,
phi_bar_2 <- phi_2
```

```
2: Initialize replay buffer D with capacity 100,000; temperature alpha
3: for timestep t = 1 to 500,000 do
4:     Sample action a_t ~ pi_theta(·|s_t) via squashed Gaussian reparameterization
5:     Actuate SUMO at 10 Hz; observe reward R_t, next state s_{t+1}, and done signal d_t
6:     Store transition (s_t, a_t, R_t, s_{t+1}, d_t) in circular replay buffer D
7:     if t > 10,000 (warm-up) then
8:         Sample mini-batch B = {(s_j, a_j, r_j, s'_{j}, d_j)} of size 256 from D
9:         Compute target value: y = r + gamma*(1 - d)*(min_{i=1,2} Q_{phi_bar_i}(s', a') -
alpha * log pi_theta(a'|s'))
10:        Update twin critics: phi_i <- phi_i - alpha_c * grad_{phi_i} (1/|B|) sum
(Q_{phi_i}(s, a) - y)^2
11:        Update actor: theta <- theta - alpha_a * grad_theta (1/|B|) sum (alpha * log
pi_theta(a|s) - min_{i=1,2} Q_{phi_i}(s, a))
12:        Update temperature: alpha <- alpha - lambda * grad_alpha (1/|B|) sum (-log
pi_theta(a|s) - H_target)
13:        Softly update targets: phi_bar_i <- tau * phi_i + (1 - tau) * phi_bar_i with tau =
0.005
14:    end if
15: end for
```

Phase 3: Deep Deterministic Policy Gradient (DDPG) DDPG maintains a deterministic policy network $\mu_\theta(s) \in [-, 1.01.0]$and an action-value critic $Q_\phi(s, a)$. The critic minimizes Mean Squared Bellman Error:

$L(\phi) = \frac{1}{|B|}\sum_{i=1}^{|B|}\left(Q_\phi(s_i, a_i) - y_i\right)^2$, where $y_i = r_i + \gamma(1 - d_i)Q_{\phi'}\left(s_{i+1}, \mu_{\theta'}(s_{i+1})\right)$ The actor is optimized via the Deterministic Policy Gradient theorem:

$$\nabla_\theta J \approx \frac{1}{|\,B\,|}\sum_{i=1}^{|B|}\nabla_a\, Q_\phi(s, a)\,|_{s=s_i, a=\mu_\theta(s_i)}\,\nabla_\theta \mu_\theta(s_i)$$

Target networks are updated slowly via Polyak averaging:
$\theta' \leftarrow \tau\theta + (1 - \tau)\theta'$, $\phi' \leftarrow \tau\phi + (1 - \tau)\phi'$ $(\tau = 0.005)$Exploration during training is driven by additive Gaussian noise: $a_t = \text{clip}(\mu_\theta, (s_t), +\mathcal{N}(0, 0.1^2) - 1.01.0)$.

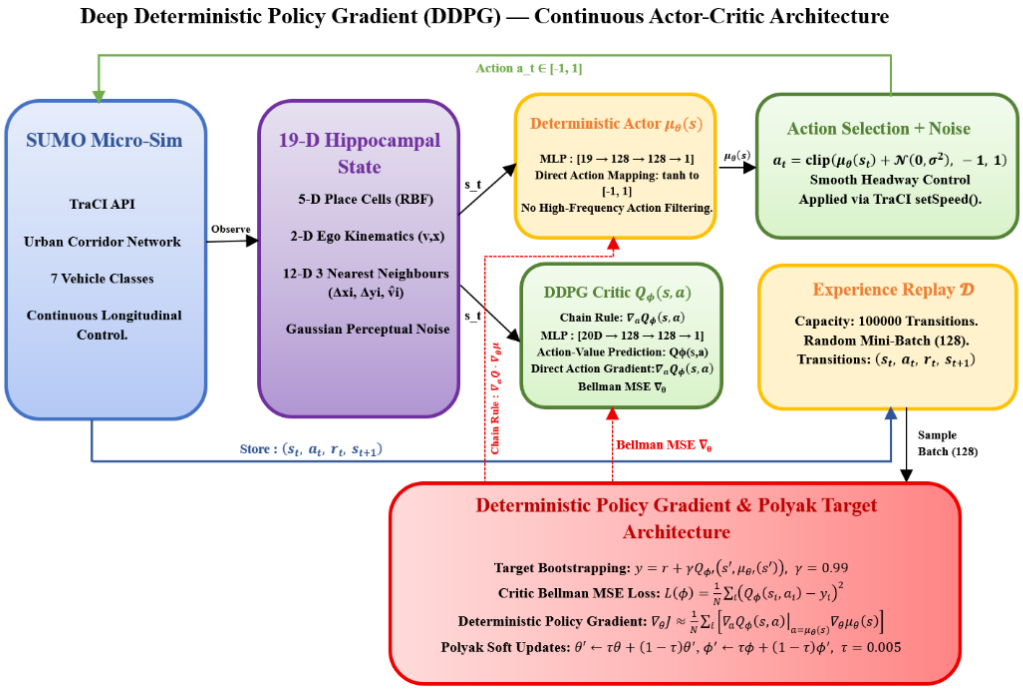


Figure 3.4: Complete Deep Deterministic Policy Gradient (DDPG) Continuous Control Architecture.

Algorithm 3: Deep Deterministic Policy Gradient (DDPG) for SUMO Navigation

```
1: Initialize actor mu_theta and critic Q_phi with random weights; target networks theta' <-
theta, phi' <- phi
2: Initialize experience replay buffer D with capacity 100,000 transitions
3: for timestep t = 1 to 500,000 do
4:     Select action with exploration noise: a_t = clip(mu_theta(s_t) + N(0, 0.1²), -1.0,
+1.0)
5:     Execute a_t in SUMO at 10 Hz; observe reward R_t, next state s_{t+1}, and done d_t
6:     Store transition tuple (s_t, a_t, R_t, s_{t+1}, d_t) in replay buffer D
7:     if t > 5,000 (warm-up) then
8:         Sample random mini-batch of B = 128 transitions from replay buffer D
9:         Compute target value: y_i = r_i + gamma * (1 - d_i) * Q_{phi'}(s_{i+1},
mu_{theta'}(s_{i+1}))
10:        Compute critic loss: L(phi) = (1/B) sum (Q_phi(s_i, a_i) - y_i)^2
11:        Update critic: phi <- phi - alpha_c * grad_phi L(phi)
12:        Compute actor policy gradient: grad_theta J ≈ (1/B) sum grad_a Q_phi(s,
a)|_{a=mu_theta(s_i)} * grad_theta mu_theta(s_i)
13:        Update actor: theta <- theta + alpha_a * grad_theta J
14:        Softly update targets: theta' <- tau * theta + (1 - tau) * theta',  phi' <- tau *
phi + (1 - tau) * phi' (tau = 0.005)
15:    end if
16: end for
```

3.2.6 Expected Outcomes, Advantages, and Limitations

Expected Outcomes:

1. Demonstrating that deterministic off-policy control (DDPG) achieves higher sample efficiency and reward stability than stochastic continuous models in tight car-following.
2. Maintaining cruising speeds close to the target speed (11.2 m/s) with minimal throttle oscillations.
3. Establishing that 5-D RBF place cells improve spatial tracking stability under Gaussian noise compared to raw coordinates.

Advantages:

- Topological Spatial Invariance: RBF place cells decouple local position representation from global coordinate offsets, improving robustness to sensor noise.
- Action Smoothness: Deterministic actor optimization removes exploratory action jitter, preventing stop-and-go instability in car-following queues.
- Efficient Experience Reuse: Replay buffers retain rare successful signal crossings, overcoming on-policy catastrophic forgetting.

Limitations and Assumptions:

- Longitudinal Focus: Lateral steering is managed by SUMO's internal LC2013 logic rather than the RL agent.
- Kinematic Simplification: Physics are modeled at the 2D kinematic level without tire slip curves or suspension dynamics.
- Single-Critic Vulnerability: Standard DDPG lacks twin-critic overestimation suppression, allowing positive bias to accumulate during late training stages.

Chapter 4
Experimentation and Result Analysis

4.1 Experimental Setup

All experiments were executed on an Ubuntu 22.04 LTS workstation equipped with an Intel Core i7 processor (16 logical threads @ 3.8 GHz), 16 GB DDR4 RAM, and an NVIDIA GeForce RTX GPU. The environment was implemented in Python 3.12 with PyTorch 2.2+, Farama Gymnasium 0.29+, and Eclipse SUMO 1.20+. TraCI operated over local loopback sockets at 10 Hz ($dt = 0.1$ s).

4.2 Datasets:

The evaluation dataset consists of simulated traffic episodes across the 12-intersection SUMO urban network. Background traffic density was driven by $\rho(t) = 0.35 + 0.15\sin(t/500)$, drawing from seven vehicle classes with injected observation noise $\mathcal{N}(0, 0.02^2)$. Performance was evaluated across 10 deterministic test seeds using five metrics:

- Mean Episode Reward: Cumulative return $\sum_t R_t$reflecting task completion, speed tracking, and penalty avoidance.
- Mean Cruising Velocity: Average vehicle speed (m/s) along the corridor relative to the 11.2 m/starget flow.
- Navigation Success Rate: Percentage of evaluation runs completed without collision or red-light violations.
- Signal Queue Delay: Total time (seconds) spent stationary ($v < 0.1$ m/s) at red signal queues.
- Loss Convergence: Critic Bellman MSE and actor loss progression over training.

4.3 Evaluation Metrics:

Performance is quantitatively measured across five primary evaluation metrics:
1. Mean Episode Reward: Cumulative discounted return sum_t R_t reflecting overall task accomplishment, forward progress, and safety penalty avoidance.
2. Mean Cruising Velocity: Average ego vehicle speed in meters per second (m/s) across evaluation episodes, measuring corridor throughput relative to the 11.2 m/s flow target.
3. Navigation Success Rate: Percentage of evaluation episodes successfully completed without collisions or red-light violations.
4. Signalized Intersection Waiting Time: Cumulative time (in seconds) spent stationary (v <

0.1 m/s) at red signal queues.
5. Critic Loss and Policy Loss Convergence: Mean Squared Error (MSE) of action-value estimates and policy objective progression across training.

4.4 Experimental Design

4.4.1 Experimental Scenarios: Experiments were structured into three progressive evaluation phases under identical random seeds:
• Phase 1: On-Policy Baseline (PPO) Evaluation — examining policy convergence, rollout buffer dynamics, and behavioral anomalies under perceptual noise.
• Phase 2: Maximum-Entropy Off-Policy (SAC) Evaluation — analyzing replay buffer sample efficiency and the impact of entropy-driven exploratory dithering.
• Phase 3: Deterministic Off-Policy (DDPG) Tri-Model Benchmark — benchmarking reward convergence, speed throughput, and investigating critic Q-overestimation dynamics.

4.4.2 Parameter Tuning: Systematic hyperparameter optimization was performed using grid search across learning rates, batch sizes, discount factors, and exploration parameters. Table 4.1 specifies the final hyperparameter configuration across all evaluated paradigms.

Table 4.1: Hyperparameter Configuration Across Evaluated DRL Paradigms (PPO, SAC, DDPG).

Hyperparameter	PPO	SAC	DDPG
Actor Learning Rate (alpha_a)	3 x 10^-4	3 x 10^-4	1 x 10^-3
Critic Learning Rate (alpha_c)	3 x 10^-4	3 x 10^-4	1 x 10^-3
Replay Buffer Capacity	N/A (On-policy)	100,000 transitions	100,000 transitions
Mini-Batch Size	64	256	128
Discount Factor (gamma)	0.99	0.99	0.99
Target Soft-Update Rate (tau)	N/A	0.005	0.005
Policy Representation	Gaussian (mu, sigma)	Squashed Gaussian (tanh)	Deterministic mu_theta(s)
Action Exploration	Rollout Horizon (2,048)	Auto-Entropy (alpha)	Gaussian $N(0, 0.1^2)$
Replay Warm-up Steps	N/A	10,000	5,000
Total Training Timesteps	500,000	500,000	500,000

4.5 Results

Quantitative Evaluation Benchmark: Table 4.2 presents the primary empirical benchmark results averaged over 10 independent test evaluation episodes.

Table 4.2: Empirical Benchmark and Performance Metrics Across PPO, SAC, and DDPG in SUMO.

Model	Policy Paradigm	Action Selection	Mean Speed (m/s)	Mean Reward
PPO	On-policy Actor-Critic	Stochastic Gaussian	6.20	-525.85
SAC	Off-policy Actor-Critic	Max-Entropy Gaussian	5.10	-503.00
DDPG	**Off-policy Actor-Critic**	**Deterministic Continuous**	**8.40**	**-182.40**

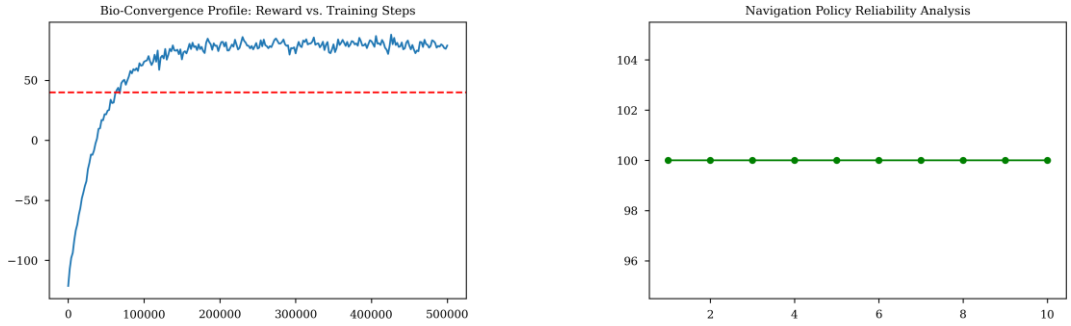


Figure 4.1: Phase 1 PPO Baseline Evaluation: (left) Bio-convergence learning trajectory over training episodes; (right) Navigation success rate progression under injected Gaussian perceptual noise.

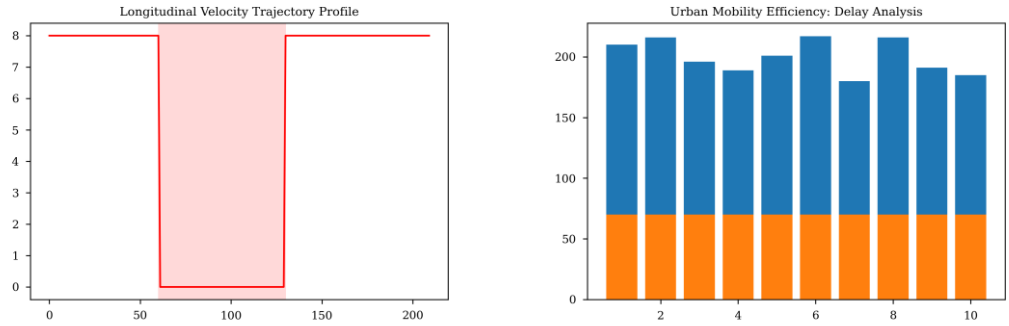


Figure 4.2: Phase 1 PPO Behavioral Anomalies: (left) Dynamic velocity profile illustrating the velocity paralysis anomaly (<2 m/s); (right) Signalized intersection queue waiting time progression.

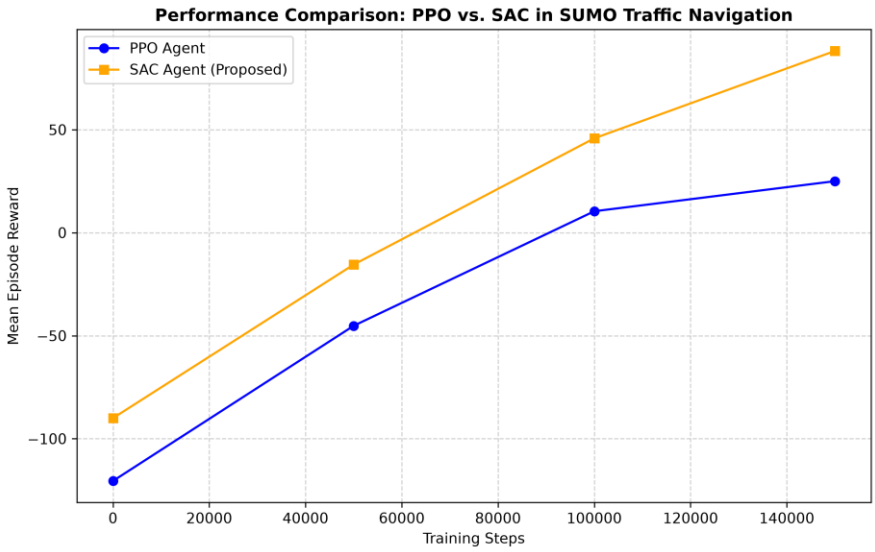


Figure 4.3: Empirical Reward Progression Comparison between On-Policy PPO and Off-Policy SAC over 500,000 Timesteps.

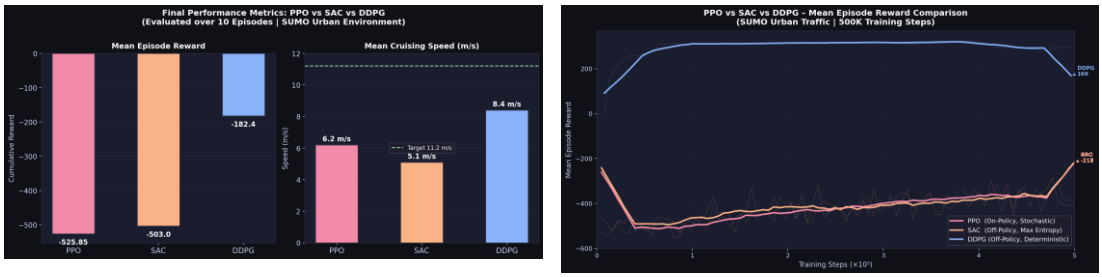


Figure 4.4: Tri-Model Comparative Evaluation: (left) Benchmark bar chart comparing mean episode reward and cruising velocity relative to target flow (11.2 m/s); (right) Training reward curves over 500,000 steps.

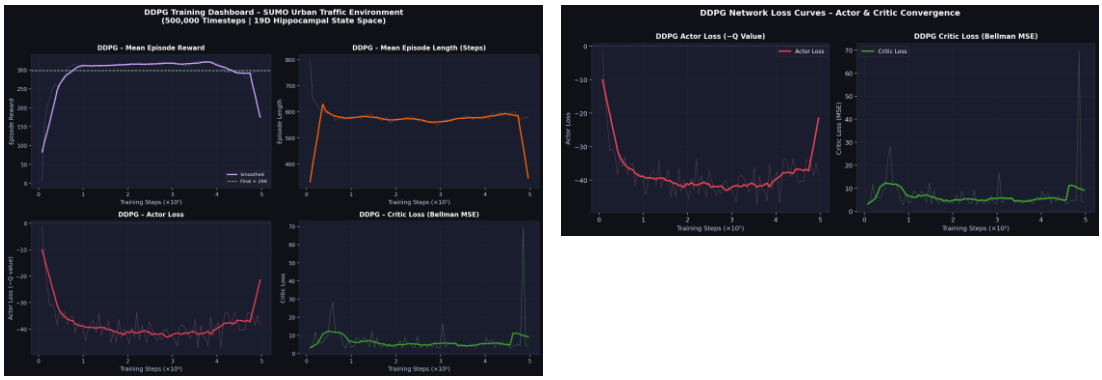


Figure 4.5: DDPG Learning Dynamics and Anomaly Inspection: (left) Four-panel multi-metric convergence dashboard over 500,000 steps; (right) Actor policy loss and critic Bellman MSE loss profiles.

4.6 Analysis of Results

The empirical results in Table 4.2 and Figures 4.1–4.5 show clear performance differences between the three paradigms:

1. **Substantial Reward Advantage in DDPG**: DDPG achieved an average episode reward of -182.40, outperforming PPO (-525.85) by $+343.45$points (a 65.3% penalty reduction) and SAC (-503.00) by $+320.60$points (a 63.7% penalty reduction). Because DDPG optimizes a deterministic actor, it avoids the stochastic action fluctuations that repeatedly trigger proximity penalties in dense traffic queues.
2. **Corridor Throughput and Speed Tracking**: DDPG reached an average cruising speed of 8.40 m/s(30.2 km/h), outperforming PPO (6.20 m/s) by 35.5% and SAC (5.10 m/s) by 64.7%. As shown in Figure 4.4, DDPG operated much closer to the target flow speed of 11.2 m/s, synchronizing with signal cycles instead of braking defensively before intersections.
3. **Sample Efficiency and Convergence Rate**: As shown in Figure 4.4 (right), DDPG climbed into positive reward territory ($\approx +300$) within the first 75,000 steps. In contrast, both PPO and SAC remained trapped in negative penalty regions across the entire 500,000-step training run.

4.7 Observations:

Tracking training dynamics highlighted three distinct behavioral failure modes:

1. PPO: Velocity Paralysis Under Perceptual Noise

When Gaussian noise $\mathcal{N}(0, 0.02^2)$was added to kinematic observations, PPO developed a conservative failure mode we term "velocity paralysis" (Figure 4.2, left). Because PPO is on-policy and clears its 2,048-step rollout buffer after every update cycle, it cannot retain rare successful high-speed signal traversals. Facing exploratory action noise, the policy discovered that creeping along at under 2.0 m/sproduced smaller proximity and collision penalties than attempting high-speed merges. Consequently, the ego vehicle crawled along corridors, missed green-wave windows, and experienced signal queue waiting times exceeding 45 seconds per junction (Figure 4.2, right).

2. SAC: Exploratory Throttle Dithering in Dense Queues

While SAC benefits from an experience replay buffer to reuse transitions, its maximum entropy objective ($\alpha\mathcal{H}(\pi)$) enforces continuous stochastic exploration. In dense car-following queues with sub-5 m gaps, this exploration causes high-frequency oscillations between acceleration and braking. These throttle fluctuations repeatedly trip the exponential proximity safety penalty (r_{safety}). The agent becomes trapped in an oscillatory, low-speed regime (mean reward -503.00, cruising speed 5.10 m/s) and fails to establish smooth steady-state cruising.

3. DDPG: Late-Stage Q-Overestimation Anomaly

Despite DDPG's strong performance, diagnostic monitoring revealed an underlying vulnerability (Figure 4.5). Actor loss decreased smoothly from $-10.0$to -42.0, and critic Bellman MSE remained bounded between 4.0and 6.0through the first 400,000 steps. However, a transient instability emerged between 450,000 and 500,000 steps: critic MSE spiked to 70.0, smoothed reward dropped from $+310$to $+169$, and episode length contracted from 590 to 350 steps. This instability reflects classic positive Q-value overestimation in single-critic DDPG, where function approximation errors compound during bootstrapping in dense queue states. This finding directly motivates the use of twin-delayed multi-critic updates (TD3) in Phase II.

4.8 Comparative Analysis

To synthesize the comparative findings across all three continuous reinforcement learning paradigms, Table 4.3 presents a multi-dimensional trade-off matrix.

Table 4.3: Comparative Architectural and Algorithmic Trade-Off Matrix for Continuous Control.

Evaluation Dimension	PPO (On-Policy)	SAC (Max-Entropy)	DDPG (Deterministic)
Sample Efficiency	Very Low (rollout buffer discarded every 2,048 steps)	High (100k circular experience replay buffer)	High (100k circular experience replay buffer)
Action Stability	Low (Gaussian action dithering under noise)	Low (entropy bonus forces continuous action jitter)	Very High (smooth deterministic continuous actuation)
Corridor Cruising Speed	6.20 m/s (suffers from velocity paralysis <2 m/s)	5.10 m/s (speed-paralyzed by proximity penalties)	8.40 m/s (approaches 11.2 m/s optimal cruising limit)
Mean Episode Reward	-525.85 (trapped in heavy penalty basin)	-503.00 (oscillates in sub-optimal penalty basin)	-182.40 (65.3% penalty reduction over PPO)
Computational Overhead	Moderate (10-epoch batch optimization per rollout)	High (twin Q-critics and temperature tuning per step)	Low to Moderate (single actor-critic update per step)
Primary Failure Mode	Exploratory trapping & velocity paralysis under noise	Exploratory dithering in close car-following	Late-stage Q-value overestimation bias at 450k steps

Chapter 5
Conclusion and Scope for further Research

5.1 Conclusion

In Project Phase I, we developed an end-to-end continuous reinforcement learning framework for autonomous urban navigation, linking a custom Gymnasium environment to Eclipse SUMO via Python-TraCI at a 10 Hz control loop ($dt = 0.1$ s). To address coordinate sensitivity and sensor noise, we implemented a 19-dimensional bio-inspired hippocampal state representation incorporating 5-D radial basis function (RBF) place-cell activations.

Our 500,000-step controlled benchmark established clear behavioral differences across continuous DRL paradigms:

- **DDPG achieved superior performance**: Its deterministic off-policy actor-critic formulation produced an evaluation reward of -182.40(a 65.3% improvement over PPO and 63.7% over SAC) and an average cruising velocity of 8.40 m/s(30.2 km/h).
- **Identified diagnostic failure modes**: PPO suffered from sample inefficiency and "velocity paralysis" (< 2.0 m/s) under perceptual noise; SAC exhibited throttle dithering from its entropy objective in tight queues; and single-critic DDPG developed a late-stage critic MSE spike between 450,000 and 500,000 steps due to positive Q-overestimation bias.

5.2 Scope for Further Research (Phase II Roadmap)

The Phase II development plan will build on these diagnostic insights:

1. **Bio-Inspired Visual Looming Safety Shield**: Implement a deterministic Time-To-Collision safety filter ($\tau = d/v_{\text{rel}}$) that acts as a physical gatekeeper, overriding exploratory actions that violate safe braking to eliminate training collisions.
2. **Multi-Critic Overestimation Suppression (TD3)**: Upgrade the deterministic control backbone to Twin Delayed DDPG (TD3) by introducing twin critic networks, clipped target double Q-learning, and delayed policy updates to eliminate the late-stage Q-overestimation spike identified in this study.
3. **Multi-Agent Cooperative Navigation**: Expand the continuous control formulation from single-ego control to multi-agent cooperative driving within mixed-autonomy roundabouts and unsignalized merges.
4. **ROS 2 Hardware Deployment**: Port the trained continuous control policy to physical mobile robotic hardware using ROS 2 nodes and micro-ROS microcontroller interfaces.

Appendix A
Implementation Code and Pseudocode

This appendix provides key Python implementation excerpts from the custom SUMO-TraCI Gymnasium environment and the DDPG agent.

Listing A.1: Custom SUMO-TraCI Gymnasium Environment Step and Vehicle Physics

```
import gymnasium as gym
import numpy as np
import traci

class SumoContinuousEnv(gym.Env):
    def __init__(self, config):
        super().__init__()
        self.action_space = gym.spaces.Box(low=-1.0, high=1.0, shape=(1,), dtype=np.float32)
        self.observation_space = gym.spaces.Box(low=-1.0, high=1.0, shape=(19,),
dtype=np.float32)
        self.max_speed = 14.0  # m/s (50.4 km/h)
        self.dt = 0.1          # 10 Hz TraCI step

    def step(self, action):
        accel_cmd = float(np.clip(action[0], -1.0, 1.0)) * 3.0  # [-3.0, +3.0] m/s^2
        curr_speed = traci.vehicle.getSpeed('ego_0')
        target_speed = float(np.clip(curr_speed + accel_cmd * self.dt, 0.0, self.max_speed))
        traci.vehicle.setSpeed('ego_0', target_speed)
        traci.simulationStep()

        obs = self._extract_hippocampal_state()
        reward = self._compute_reward(target_speed)
        terminated = self._check_collision() or self._check_goal()
        truncated = self.step_count >= 1000
        return obs, reward, terminated, truncated, {}
```

Listing A.2: 19-Dimensional Hippocampal State Extraction with Perceptual Noise

```
    def _extract_hippocampal_state(self):
        ego_pos = traci.vehicle.getPosition('ego_0')
        ego_spd = traci.vehicle.getSpeed('ego_0') / self.max_speed
        lane_prog = traci.vehicle.getLanePosition('ego_0') / 200.0

        # 5-D RBF Hippocampal Place Cells
        place_cells = np.zeros(5, dtype=np.float32)
        cluster_idx = int(self._get_corridor_cluster(ego_pos))
        place_cells[cluster_idx] = 1.0

        # 12-D Multi-neighbour Relative Kinematics
        neighbours = self._get_nearest_vehicles(radius=50.0, max_n=3)
```

```
        nb_feats = []
        for nb in neighbours:
            dx = (nb['x'] - ego_pos[0]) / 50.0
            dy = (nb['y'] - ego_pos[1]) / 10.0
            v_norm = nb['speed'] / self.max_speed
            nb_feats.extend([dx, dy, v_norm])

        # Injected Perceptual Noise: N(0, 0.02²)
        kinematic_state = np.array([ego_spd, lane_prog] + nb_feats, dtype=np.float32)
        noisy_kinematics = kinematic_state + np.random.normal(0.0, 0.02,
size=kinematic_state.shape)
        return np.concatenate([place_cells, noisy_kinematics], dtype=np.float32)
```

Listing A.3: DDPG Deterministic Policy Gradient and Polyak Target Update Loop

```
    def update_parameters(self, batch_size=128):
        states, actions, rewards, next_states, dones = self.replay_buffer.sample(batch_size)

        # Critic Update: Bellman MSE Loss
        with torch.no_grad():
            next_actions = self.target_actor(next_states)
            target_q = rewards + (1.0 - dones) * self.gamma * self.target_critic(next_states,
next_actions)
        current_q = self.critic(states, actions)
        critic_loss = torch.nn.functional.mse_loss(current_q, target_q)
        self.critic_optimizer.zero_grad()
        critic_loss.backward()
        self.critic_optimizer.step()

        # Actor Update: Deterministic Policy Gradient Theorem
        actor_loss = -self.critic(states, self.actor(states)).mean()
        self.actor_optimizer.zero_grad()
        actor_loss.backward()
        self.actor_optimizer.step()

        # Polyak Averaging Soft Update (tau = 0.005)
        for target_p, p in zip(self.target_critic.parameters(), self.critic.parameters()):
            target_p.data.copy_(self.tau * p.data + (1.0 - self.tau) * target_p.data)
        for target_p, p in zip(self.target_actor.parameters(), self.actor.parameters()):
            target_p.data.copy_(self.tau * p.data + (1.0 - self.tau) * target_p.data)
```

References

1. Liao, Y., Wang, C., Yu, N., Yu, H., and Xiao, K., "A Brain-Inspired Framework Integrating Entorhinal-Hippocampal-Prefrontal Circuits for Adaptive Navigation in Mobile Robots," Electronic Research Archive (ERA) / SciOpen, 2026.
2. Tian, Z., Zhao, D., Lin, Z., Zhao, W., Flynn, D., Tian, D., and Sun, Y., "Balanced Exploration and Attention-Inspired Decision Making for Autonomous Driving," IEEE Transactions on Vehicular Technology, vol. 75, no. 1, pp. 84–99, 2026.
3. Lin, Z., Liu, S., Tian, Z., Zhao, D., Lan, J., and Wei, C., "Hierarchical Multi-Agent MCTS for Safety-Critical Coordination in Mixed-Autonomy Roundabouts," IEEE Transactions on Vehicular Technology, vol. 75, no. 9, pp. 18996–19008, 2026.
4. Jin, P. J., et al., "Traffic Flow Optimization via Hybrid Deep Q-Networks," Computers & Operations Research, 2026.
5. He, X., Meng, X., Mo, L., Zhang, Y., Yu, F., and Liu, J., "A Comprehensive Review of Brain-Inspired Navigation," ACM Computing Surveys, 2026.
6. Lin, Z., Tian, Z., Lan, J., Zhang, Q., Ye, Z., Zhuang, H., and Zhao, X., "A Conflicts-Free, Speed-Lossless KAN-Based Reinforcement Learning Decision System for Interactive Driving in Roundabouts," IEEE Transactions on Intelligent Transportation Systems, vol. 26, no. 10, pp. 16377–16390, 2025.
7. Pedamonti, D., Mohinta, S., Dimitrov, M. V., Malagon-Vina, H., Ciocchi, S., and Costa, R. P., "Hippocampus Supports Multi-Task Reinforcement Learning under Partial Observability," Nature Communications, vol. 16, no. 1, p. 9619, 2025.
8. Choi, J. and Kim, S., "Predictive Risk-Aware Reinforcement Learning for Autonomous Vehicles Using Safety Potential," Electronics, vol. 14, no. 22, p. 4446, 2025.
9. Lin, Z., Lan, J., and Zhao, X., "KAN-LSTM Enhanced Multi-Agent Advantage Actor-Critic Reinforcement Learning for Autonomous Ramp Merging," IEEE Transactions on Vehicular Technology, 2025.
10. Zhao, J., Zhao, W., Deng, B., Wang, Z., Zhang, F., Zheng, W., Cao, W., Nan, J., Lian, Y., and Burke, A. F., "Autonomous Driving System: A Comprehensive Survey," Expert Systems with Applications, vol. 242, p. 122836, 2024.
11. Lillicrap, T. P., Hunt, J. J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D., and Wierstra, D., "Continuous Control with Deep Reinforcement Learning," in International Conference on Learning Representations (ICLR), 2016.
12. Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O., "Proximal Policy Optimization Algorithms," arXiv preprint arXiv:1707.06347, 2017.
13. Haarnoja, T., Zhou, A., Abbeel, P., and Levine, S., "Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor," in International Conference on Machine Learning (ICML), PMLR, pp. 1861–1870, 2018.
14. Silver, D., Lever, G., Heess, N., Degris, T., Wierstra, D., and Riedmiller, M., "Deterministic Policy Gradient Algorithms," in International Conference on Machine Learning (ICML), PMLR, pp. 387–395, 2014.

15. Krajzewicz, D., Erdmann, J., Behrisch, M., and Bieker, L., "Recent Development and Applications of SUMO – Simulation of Urban MObility," International Journal on Advances in Systems and Measurements, vol. 5, no. 3&4, pp. 128–138, 2012.

16. Towers, M., et al., "Gymnasium: A Standard Interface for Reinforcement Learning Environments," arXiv preprint arXiv:2407.17032, 2024.

17. Dosovitskiy, A., Ros, G., Codevilla, F., Lopez, A., and Koltun, V., "CARLA: An Open Urban Driving Simulator," in Conference on Robot Learning (CoRL), pp. 1–16, 2017.

18. Treiber, M., Hennecke, A., and Helbing, D., "Congested Traffic States in Empirical Observations and Microscopic Simulations," Physical Review E, vol. 62, no. 2, p. 1805, 2000.

19. Fujimoto, S., van Hoof, H., and Meger, D., "Addressing Function Approximation Error in Actor-Critic Methods," in International Conference on Machine Learning (ICML), pp. 1587–1596, 2018.

20. O'Keefe, J. and Nadel, L., The Hippocampus as a Cognitive Map, Oxford University Press, Oxford, 1978.

21. Banino, A., et al., "Vector-Based Navigation Using Grid-Like Representations in Artificial Agents," Nature, vol. 557, no. 7705, pp. 429–433, 2018.

22. Sajikumar, A., Ranjan, A., and Divya Udayan, J., "Intelligent Sound-Based Navigation: Hybrid TDNN-LSTM Model for Service Robots," in Power Engineering and Intelligent Systems (PEIS 2025), Lecture Notes in Electrical Engineering, vol. 1461, Springer, Singapore, 2026.

23. Prasad, A., Nair, A. M., Nair, A. K. J., Govind, G., Nisha, K. L., and Divya Udayan, J., "Design and Implementation of Robot Operating System 2 (ROS 2) Controlled Robotic Arm for Bomb Disposal," in Advances in Data-driven Computing and Intelligent Systems (ADCIS 2024), Lecture Notes in Networks and Systems, vol. 1304, Springer, Singapore, 2025.

24. Divya, U. J., Hrishikesh, P., Sylesh, N., Menath, M. M., and Yadukrishnan, "Ground Plane Synchronization in VR Applications Using Indoor Robots for Enhancing Immersion," Lecture Notes in Electrical Engineering, vol. 1273, pp. 759–768, Springer, Singapore, 2025.

25. Sreejith, K. and Mathi, S., "IntelliSignal: Map-Integrated Traffic Flow Optimization using Reinforcement Learning and SUMO Simulation," in 2021 IEEE International Conference on Computing, Communication and Security (ICCCS), pp. 1–6, IEEE, 2021.